NATIONAL POLYTECHNIC UNIVERSITY OF ARMENIA

Faculty of Applied Mathematics and Physics

Optimal Allocation of Energy Resources of a Grid Connected Microsystem: Nonlinear Case

Faculty/institute: Informatics and Applied Mathematics (Computer Science)

Group: MT240a

Student: Vany Kardjikian

Supervisor: I. Hovhannisyan

YEREVAN 2026

Table of Contents

Introduction.....3

Chapter 1: Literature Review.....4

1.1. Study of Existing Models..... 4

1.2 Definitions of Key Concepts:..... 4

1.3. Optimization Algorithms Relevant to the Problem:..... 5

1.3.1 Simplex..... 5

1.3.2 Sequential quadratic programming:.....6

1.3.3 Branch and bound..... 8

1.3.4 Branch-and-Cut Method..... 10

1.3.5 Branch-and-Bound Method for Mixed-Integer Nonlinear Programming (MINLP).11

1.4 Tools and Software..... 13

1.4.1 Python..... 13

1.4.2 Libraries..... 13

Chapter 2: Problem Statement and Mathematical Formulation..... 16

2.1 Problem Statement.....16

2.2 Linear Programming Model..... 18

2.2.1 Mathematical Model..... 18

2.2.2 Solution Methodology:..... 20

2.3. Nonlinear Programming Model:..... 22

2.3.1 Mathematical Model:.....22

2.3.2 Solution Methodology:..... 22

Chapter 3: Software Implementation and Code Structure.....24

3.1 Code Structure:..... 24

Chapter 4: Case Study: Commercial Building in Yerevan..... 28

4.1 System parameters..... 28

4.2 Results.....30

4.3 Discussion.....34

Conclusion..... 42

References.....43

Introduction

Armenia is experiencing rapid growth in distributed solar generation. Currently, thousands of commercial and residential buildings in Armenia have rooftop photovoltaic systems installed.

At the same time, time-of-use tariffs for commercial consumers have introduced a significant price difference between night and day, while export compensation for surplus solar energy remains low.

Adding a lithium-ion battery to the system allows the building owner to store excess midday solar production and either use it during expensive peak hours or sell it back to the grid.

Achieving the maximum possible daily profit requires making the correct decision every single hour of the day: charge, discharge, buy from the grid, or sell to the grid.

This decision problem is complex because:

- The battery cannot charge and discharge at the same time,
- Real inverters and batteries have losses that increase quadratically with power.

The goal of this diploma thesis is to develop mathematical optimisation models that find the absolute best 24-hour schedule for a typical commercial building in Yerevan under 2025 tariffs, and to quantify how much money can be saved when realistic nonlinear losses are taken into account compared to the simplified linear models commonly used in practice.

All calculations are performed using only open-source Python tools: PuLP + CBC for the linear case and GEKKO (APOPT) for the nonlinear case.

Chapter 1: Literature Review

1.1. Study of Existing Models

Optimal daily energy management of PV-battery systems under time-of-use tariffs has been a widely studied problem. Most works use mixed-integer linear programming (MILP) with constant efficiency assumption and achieve cost reduction compared to grid-only operation.

Recent studies show that real inverter and battery losses are quadratic in power and ignoring them overestimates savings. Such problems are formulated as convex mixed-integer nonlinear programs (MINLP) and solved to proven global optimality using approximation algorithms implemented in tools like GEKKO.

This thesis develops and compares linear and nonlinear models using open-source tools, and quantifies the impact of quadratic losses for a commercial building in Yerevan under different export compensation scenarios.

1.2 Definitions of Key Concepts:

Mathematical Programming Problem:

A mathematical programming problem consists of finding values of the decision variables that minimize (or maximize) an objective function, subject to a set of constraints.

$$\begin{cases} f(\vec{x}) \to \min(\max), \\ g_i(\vec{x}) = 0, \ i = \overline{1, m}, \end{cases} \tag{1}$$

Linear Programming Problem:

A linear programming problem is a mathematical optimization problem in which the objective function and all constraints are linear functions of the decision variables.

Nonlinear Programming Problem:

A nonlinear programming problem is a mathematical optimization problem in which at least one of the objective function or constraints is nonlinear.

Mixed-Integer Programming (MIP) Problem:

A mixed-integer programming (MIP) problem is a mathematical optimization problem in which some of the decision variables are required to take integer values, while the remaining variables may be continuous.

Gradient of a function:

For a given function f, the first order gradient is the following vector:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}(\mathbf{x}), \frac{\partial f}{\partial x_2}(\mathbf{x}), \cdots, \frac{\partial f}{\partial x_n}(\mathbf{x}) \right)$$

Lagrangian Function:

Given the problem (1), the lagrangian function is defined as:

$$L(\vec{x}, \vec{\lambda}) = f(\vec{x}) + \lambda_1 g_i(\vec{x}) + \lambda_2 g_2(\vec{x}) + \cdots + \lambda_m g_m(\vec{x})$$

1.3. Optimization Algorithms Relevant to the Problem:

1.3.1 Simplex

The Simplex algorithm, introduced by George Dantzig, is a foundational method for solving linear programming (LP) problems. It systematically searches the vertices of the feasible region, which is a convex polyhedron defined by linear constraints, to find the optimal value of a linear objective function.

Two key properties of LP form the theoretical basis of the Simplex method:

1. **Convexity of the feasible region:** The set of feasible solutions in an LP problem is convex, meaning any optimal solution lies at an extreme point (vertex) of the feasible polyhedron.

2. **Correspondence of basic feasible solutions and extreme points:** Each extreme point of the feasible region corresponds to a basic feasible solution, which is defined by selecting a set of basic variables while keeping the remaining non-basic variables at zero.

Algorithm

The Simplex method operates through iterative pivoting between dictionaries, each representing a basic feasible solution:

- **Entering and leaving variables:** An entering variable is selected from the non-basic variables to increase (or decrease, in minimization) the objective function. A leaving variable is selected from the basic variables to maintain feasibility (non-negativity).

- **Pivot operation:** The selected entering and leaving variables are exchanged through row operations to update the dictionary. This preserves feasibility and progresses toward the optimal solution.

- **Optimality criterion:** Iteration continues until no further improvement is possible, which occurs when all coefficients in the objective function row of the tableau are non-negative (for maximization problems).

1.3.2 Sequential quadratic programming:

Sequential quadratic programming (SQP) is a class of algorithms that can solve nonlinear optimization problems (NLP) in the real world. It can handle any degree of nonlinearity including nonlinearity in the constraints.

$$
\begin{aligned}
&f(\mathbf{x}) \longrightarrow \boldsymbol{min} \\
&g_i(\mathbf{x}) \geq 0 \quad i = 1,\ldots,k \\
&g_i(\mathbf{x}) = 0 \quad i = k+1,\ldots,m
\end{aligned}
$$

In order to make the problem easier to solve, a QP approximation is a method used where the nonlinear optimization problem is approximated by a simpler quadratic problem at each iteration.

The quadratic approximation at the point x^k is:

$$
\begin{aligned}
&f_a = f^k + \left(\nabla f^k\right)^{\mathrm{T}} \Delta\mathbf{x} + \frac{1}{2}\left(\Delta\mathbf{x}\right)^{\mathrm{T}} \nabla_x^2 \mathrm{L}^k \Delta\mathbf{x} \quad \longrightarrow min \\
&g_i^k + \left(\nabla g_i^k\right)^T \Delta\mathbf{x} \geq 0 \quad i = 1,2,\ldots,k \\
&g_i^k + \left(\nabla g_i^k\right)^T \Delta\mathbf{x} = 0 \quad i = k+1,\ldots,m
\end{aligned}
$$

Now to solve the new problem, we will solve the KKT equations for this problem.

$$\nabla f - \sum_{i=1}^{m} \lambda_i \nabla g_i = 0$$

$$g_{i,a} = 0 \quad i = 1,2,\ldots,m$$

SQP Algorithm Procedure:

a. Initialization

The algorithm begins with an initial estimate x^0 of the decision variables, which serves as the starting point for the iterative optimization process.

b. Quadratic Programming (QP) Approximation

A quadratic programming approximation of the original nonlinear optimization problem is constructed. In the first iteration, the Hessian matrix of the Lagrangian is initialized as the identity matrix.

c. Solution of the QP Subproblem

The resulting QP subproblem is solved to obtain a search direction. During this step, the Lagrange multipliers are also computed.

d. Line Search Procedure

A line search is performed along the computed search direction to ensure sufficient improvement in the objective function. The step is defined as:

$$\mathbf{x}^{new} = \mathbf{x}^{old} + \Delta\mathbf{x}$$

Where Δx is the initial step.

A penalty function is evaluated:

$$P = f + \sum_{i=1}^{vio} \lambda_i \left| g_i \right|$$

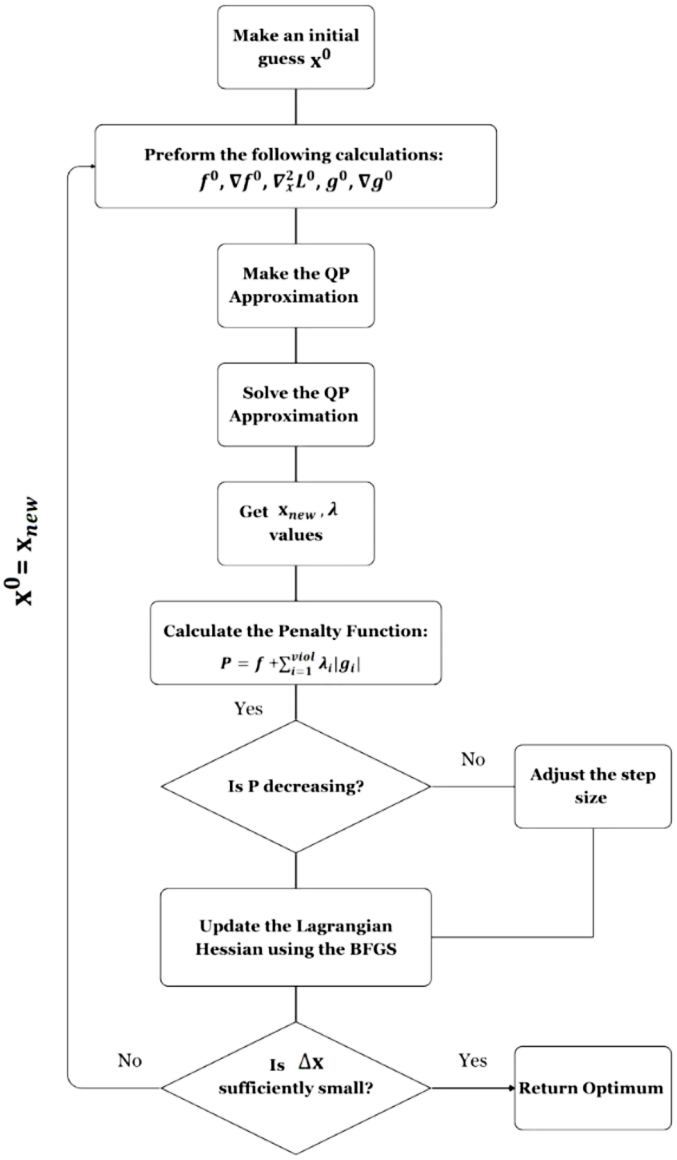
where the summation is taken over the violated constraints. If the penalty function does not decrease, the step size is reduced until a decrease is achieved.

e. Evaluation and Hessian Update

The gradient of the Lagrangian is evaluated at the new iterate. The differences in the decision variables and in the Lagrangian gradients are computed. The Hessian approximation of the Lagrangian is then updated using the BFGS (Broyden–Fletcher–Goldfarb–Shanno) method.

f. Iterative Process

Steps 2 through 5 are repeated until the step Δx becomes sufficiently small. As Δx approaches 0, the Karush–Kuhn–Tucker (KKT) optimality conditions are satisfied, the optimal solution has been found.



1.3.3 Branch and bound

Branch and Bound is a systematic tree-based method for solving discrete and combinatorial optimization problems, including mixed-integer linear programming (MILP) and mixed-integer nonlinear programming (MINLP). The method constructs a state space tree, where each node represents a partial solution obtained by fixing a subset of decision variables.

The algorithm proceeds through three fundamental operations:

- **Generation of the State Space Tree (Branching):**
  The original problem is decomposed into subproblems by successively fixing integer decision variables. Each branching step creates child nodes corresponding to alternative

decisions (e.g., $x_i = 0$ or $x_i = 1$ in binary problems). All children of an expanded node are generated before the algorithm proceeds to explore other live nodes, forming a structured search tree.

- **Evaluation of Candidate Nodes (Bounding):**
  Each node is evaluated by solving a relaxed version of the corresponding subproblem. The relaxation provides a bound on the best achievable objective value within that branch. In minimization problems, the relaxation yields a lower bound and in maximization problems, it yields an upper bound.
  The bounding process relies on computing the bound of each node (via relaxation of integrality constraints) and the current best-known feasible solution (incumbent), which serves as a reference for comparison.

- **Pruning (Node Elimination):**
  A node becomes inactive (dead) if its relaxation is infeasible, its bound is worse than the incumbent solution, or the relaxation solution is integer-feasible. This pruning mechanism reduces computational effort by eliminating subproblems that cannot improve the objective value.

Search Strategies within the State Space Tree

There are different methods based on the order in which nodes are explored. These strategies influence computational efficiency but do not alter correctness.

- **FIFO (First-In-First-Out):**
  Uses a queue structure and performs breadth-first search, exploring nodes level by level.

- **LIFO (Last-In-First-Out):**
  Uses a stack structure and performs depth-first search, exploring the most recently generated node first.

- **Least-Cost (Best-Bound) Strategy:**
  Selects the node with the most promising bound (e.g., smallest lower bound in minimization). Nodes whose cost exceeds a predefined upper bound are discarded without further expansion. This strategy is particularly important in mathematical programming solvers, as it prioritizes nodes that are more likely to contain the optimal solution.

1.3.4 Branch-and-Cut Method

The branch-and-cut (B&C) method is a powerful deterministic algorithmic framework for solving mixed-integer linear programs (MILPs) to proven global optimality. It extends the classical branch-and-bound (B&B) algorithm by dynamically incorporating cutting planes (valid inequalities) to strengthen the linear programming (LP) relaxations at nodes of the search tree. This tightening significantly reduces the number of nodes explored compared to pure branch-and-bound, making B&C the dominant approach in modern MILP solvers.

Branch-and-cut is primarily designed for problems where the objective function and all constraints are linear, and a subset of variables is restricted to integer values (MILP). It also applies to pure integer programs (IP) and many structured combinatorial optimization problems, such as the traveling salesman problem (TSP), set covering, or vehicle routing.

Algorithm:

Branch-and-cut operates on a search tree where each node represents a subproblem obtained by adding integrality constraints and/or additional bounding constraints to the original problem. The key innovation over standard branch-and-bound lies in the iterative generation and addition of cutting planes:

- At each node, an LP relaxation is yielded.
- The LP is solved (typically via the simplex method or interior-point methods) to obtain a lower bound on the optimal value in that subproblem and a (possibly fractional) solution.
- If the LP solution is integer-feasible and satisfies all original constraints, it provides an upper bound (incumbent solution) for the overall problem.
- If the solution is fractional, the algorithm identifies violated valid inequalities, linear constraints satisfied by all integer-feasible points but violated by the current fractional solution.
- These cuts are added to the current LP relaxation, and the strengthened LP is re-solved.
- The cutting process repeats until no violated cuts are found (or a separation limit is reached) or the solution becomes integer.
- If the final LP solution remains fractional, the algorithm branches on a fractional variable (typically by imposing bounds, creating two child subproblems.
- Standard bounding and pruning rules apply: subregions are fathomed if their lower bound exceeds the current incumbent upper bound or if infeasibility is proven.

Types of Cutting Planes:

A wide variety of cutting-plane families are used in practice, selected based on problem structure, separation efficiency, and bound improvement:

- **Gomory mixed-integer cuts**: Derived from the simplex tableau; applicable in general MILPs.
- **Cover cuts** (knapsack cover, lifted cover): For knapsack-like constraints.
- **Clique cuts**: Based on conflict graphs or clique inequalities.
- **Flow cuts** (e.g., flow cover, mixed-integer rounding): For network-flow or capacity-constrained structures.
- **Problem-specific cuts**: Tailored inequalities (e.g., subtour elimination for TSP, comb inequalities).

Implementations distinguish between:

- **Global cuts**: Valid across the entire tree (added at the root and propagated).
- **Local cuts**: Valid only within a subtree (stronger but more expensive to manage).
- **Lazy cuts**: Generated only when needed (e.g., for user-defined or complex constraints).

1.3.5 Branch-and-Bound Method for Mixed-Integer Nonlinear Programming (MINLP)

The branch-and-bound method was originally proposed by Land and Doig (1960) for discrete programming problems. It has been extended to solve mixed-integer nonlinear programs (MINLPs), which combine continuous variables subject to nonlinear constraints with discrete (typically binary or integer) variables. In engineering design and process systems applications, binary variables often model discrete decisions (e.g., whether to include a unit, select a material, or activate a design option), while continuous variables represent operational parameters (e.g., flows, sizes, temperatures).

The general MINLP solved by branch-and-bound takes the following form where f and the components of g and h are continuously differentiable, but potentially nonconvex.

$$
\begin{aligned}
\min_{x,y} \quad & f(x,y) \\
\text{subject to} \quad & g(x,y) \le 0, \\
& h(x,y) = 0, \\
& x \quad \in [x^L, x^U] \subseteq \mathbb{R}^n, \\
& y \quad \in \{0,1\}^m \quad (\text{or } y \in \mathbb{Z}^m \text{ in more general cases}).
\end{aligned}
$$

Branch-and-bound for MINLP relies on two fundamental operations: relaxation (to obtain bounds) and search (via branching on discrete variables). The method builds a search tree where each node corresponds to a subproblem with fixed or tightened bounds on some integer variables.

- **Relaxation**: The integrality constraints on y are temporarily ignored, yielding a relaxed MINLP (RMINLP) or, in convex cases, a nonlinear program (NLP). Solving the relaxation provides a lower bound on the optimal value of the subproblem. If the relaxed solution is integer-feasible, it yields a valid upper bound (incumbent solution) for the original MINLP.
- **Branching**: If the relaxed solution has fractional integer variables, the node is branched by fixing a fractional y_k to 0 in one child node and to 1 in the other (or more generally, imposing integer bounds). This partitions the feasible region and creates subproblems.
- **Bounding and Pruning**: Subproblems are fathomed if their lower bound exceeds the current incumbent upper bound, if they are proven infeasible, or if integrality is achieved.
- **Enhancements**: For efficiency, Gomory mixed-integer cuts are added as valid inequalities to tighten the feasible region, reducing the number of nodes and NLP subproblems solved. These cuts serve as an alternative or complement to branching.

Algorithm:

1. **Initialization** Set upper bound $U=+\infty$ and lower bound $L=-\infty$. Initialize binary variables. Solve the NLP relaxation with fixed y. if feasible, obtain an initial feasible solution and upper bound; otherwise, update y and repeat until feasibility or prove infeasibility of the MINLP.
2. **Solving the Relaxed Problem at Nodes** For a selected node with fixed or partial integer assignments, solve the corresponding NLP relaxation to obtain a lower bound.

3. **Node Processing** Select an unsolved node from the list. Solve the NLP subproblem. If feasible and integer solution found: update incumbent U and best solution if improved. If fractional, generate Gomory mixed-integer cuts (if violated) to tighten the relaxation, or branch on a fractional integer variable. Add child nodes to the list and continue.
4. **Termination** The algorithm terminates when no unsolved nodes remain. The best incumbent solution is the global optimum (for convex MINLPs) or a rigorously bounded solution (for nonconvex cases with appropriate relaxations).

For convex MINLPs, the NLP relaxation is solved to global optimality, providing valid lower bounds. For nonconvex MINLPs, additional convex underestimators (e.g., convex envelopes or spatial branching on continuous variables) are required to ensure reliable lower bounds.

1.4 Tools and Software

1.4.1 Python

Python is a programming language created by Guido van Rossum in 1991. It works on different platforms and has a simple syntax similar to the English language.

Python can be used in many domains:

- Web and Internet Development
- Scientific and Numeric
- Education
- Desktop GUIs
- Software Development

1.4.2 Libraries

Pandas

Pandas is a Python library used to analyze data. It provides fast and flexible tools to work with data. It integrates smoothly with other libraries such as Matplotlib and NumPy.

Matplotlib

Matplotlib is a library for creating static, animated, and interactive visualizations in Python.

PuLP

PuLP is a linear and mixed integer programming modeler. It can be used to create MILP optimization problems and solve them with the latest solvers. PuLP can generate MPS or LP

files and call solvers such as GLPK, COIN-OR CLP/CBC, CPLEX, GUROBI, MOSEK, XPRESS, CHOCO, MIPCL, HiGHS, SCIP/FSCIP.

PuLP employs the COIN-OR Branch and Cut (CBC) solver as its default solver for mixed-integer programming problems. CBC is an open-source mixed-integer program (MIP) solver written in C++, developed as part of the COIN-OR project and led historically by contributors such as John Forrest.

CBC is designed primarily as a callable library to support the implementation of customized branch-and-cut algorithms, although a standalone executable version is also provided. It relies on other COIN-OR components: the CLP solver for linear programming relaxations, the OSI (Open Solver Interface) for communication with LP solvers, and the CGL (Cut Generation Library) for generating cutting planes.

The core algorithm implemented by CBC is branch-and-cut (more precisely, branch-cut-and-bound). It begins by solving the LP relaxation of the MIP (treating integer variables as continuous) to obtain a lower bound on the objective. If the relaxation solution is already integer-feasible, the process terminates with an optimal solution. Otherwise, CBC branches on a fractional integer variable, creating child subproblems with tightened bounds on that variable, and adds the resulting nodes to a search tree.

At selected nodes (particularly the root), CBC applies cutting planes from CGL to strengthen the LP relaxations by adding valid inequalities that exclude the current fractional solution without removing feasible integer points. This tightening often improves bounds and enables earlier pruning of branches. Pruning occurs when a node is infeasible, when its LP bound is worse than the best-known integer solution (upper bound for minimization), or when an integer-feasible solution is found that updates the incumbent.

CBC continues exploring the search tree until optimality is proven (lower and upper bounds coincide) or a termination criterion is met.

GEKKO:

GEKKO is a Python library that can be used for machine learning and optimization of mixed-integer and differential algebraic equations. It is integrated with large-scale solvers capable of handling linear programming (LP), quadratic programming (QP), nonlinear programming (NLP), mixed-integer linear programming (MILP), and mixed-integer nonlinear programming (MINLP).

The software supports multiple applications, including steady-state and dynamic simulation, parameter estimation, data reconciliation, real-time optimization, and nonlinear predictive control.

GEKKO provides a high-level framework for formulating mathematical models. In this structure, model components are defined as constants, parameters, and variables, which are connected through algebraic equations or intermediate expressions. Objective functions are specified to minimize or maximize selected performance criteria. The software also includes built-in objects that group related variables, equations, and objectives, supporting modular and object-oriented model development.

The computational engine behind GEKKO is APMonitor, which compiles models into byte-code and performs structural analysis to reduce problem size based on sparsity patterns.

GEKKO offers nine operational modes, grouped into steady-state and dynamic categories. Steady-state modes (simulation, parameter update, and real-time optimization) assume zero derivatives. Dynamic modes (simulation, estimation, and control) describe time-dependent system behavior using differential equations. Each dynamic mode can be solved using either a simultaneous or a sequential numerical approach.

APMonitor supplies nonlinear programming solvers (such as APOPT, IPOPT, SNOPT, MINOS, and BPOPT) with the necessary model information in sparse matrix form. This includes variable definitions and constraints, equations, objective functions, sparsity structure, residual evaluations, and first- and second-order derivative information (gradients and Hessians). After the solution, results are exported in a structured format and returned to the GEKKO interface.

If convergence is not achieved, diagnostic reports identify infeasible equations and potential modeling issues. Additional debugging options allow users to increase diagnostic output or decompose the model into smaller subsets to isolate inconsistencies and improve initialization.

Chapter 2: Problem Statement and Mathematical Formulation

2.1 Problem Statement

Modern commercial and residential buildings increasingly combine rooftop photovoltaic systems, battery energy storage, and grid connection under time-of-use tariffs and net-billing regulations. In the Armenian context of 2025/2026, such buildings face the challenge of optimally managing energy flows on an hourly basis to minimize electricity costs (or maximize financial benefit).

The core decision problem is to determine, for each hour of the day, the most economically advantageous operating strategy given:

- Time-varying electricity demand of the building
- Deterministic hourly solar generation from the rooftop PV system
- Time-of-use grid purchase prices and regulated export prices
- A lithium-ion battery with limited capacity, charge/discharge power, and round-trip efficiency

The system must decide:

- Quantity of electricity to purchase from or sell to the grid
- Battery charge and discharge rates

while respecting:

- Mutual exclusivity of simultaneous battery charging/discharging
- State of charge dynamics and power limits

Objective: maximize daily electricity cost savings compared to a baseline without local generation/storage.

The problem is solved in two variants:

1. Mixed-integer linear program (MILP) using PuLP/CBC
2. Convex mixed-integer nonlinear program (MINLP) using GEKKO with Outer Approximation (APOPT).

The studied system consists of four main components connected through a single hybrid inverter:

- Rooftop photovoltaic panels (DC output)
- Lithium-ion battery storage (bidirectional DC)
- Building electrical load (AC)
- Utility grid connection operating under net-billing rules (bidirectional AC)

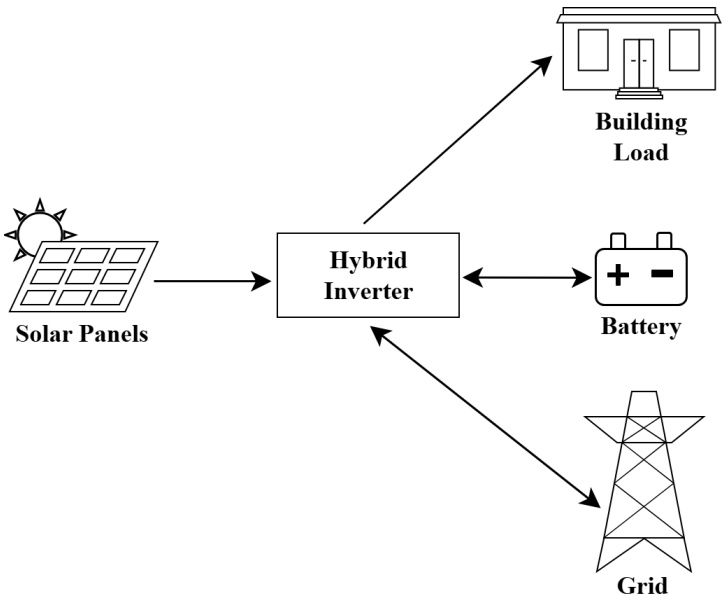


Figure 1. Schematic diagram of a grid-connected photovoltaic system with battery energy storage and hybrid inverter

All energy flows are managed centrally by the hybrid inverter, which enables simultaneous solar generation, battery charging or discharging, and grid import or export while respecting the mutual-exclusivity constraints described below.

The linear model assumes constant efficiency while the nonlinear model includes quadratic loss terms. Although the nonlinear formulation better reflects physical reality, the linear case serves as an important benchmark and is significantly easier to solve and analyse.

Therefore, to fully understand the more complex nonlinear case and to clearly quantify the impact of realistic losses, we first study the linear formulation before extending it to the nonlinear case.

2.2 Linear Programming Model

2.2.1 Mathematical Model

Indices

$t = 0, 2, 3, \dots, T$	Hours in the optimization period

Parameters

C_t^{buy}	Electricity purchase price at hour t (AMD/kWh)
C_t^{sell}	Electricity selling price at hour t (AMD/kWh)
E_t^{solar}	Solar generation at hour t (kWh)
E_t^{demand}	Load demand at hour t (kWh)
E^{cap}	Battery capacity (kWh)
$P_{charge}^{max}, P_{discharge}^{max}$	Max battery charge/discharge power (kW)
$P_{buy}^{max}, P_{sell}^{max}$	Max grid buy/sell power (kW)
$\eta_{charge}, \eta_{discharge}$	Battery efficiency
$s_{initial}$	Initial battery state of charge

Variables

x_t^{buy}	Energy bought from the grid	(Continuous)
x_t^{sell}	Energy sold to the grid	(Continuous)
x_t^{charge}	Energy charged into battery	(Continuous)
$x_t^{discharge}$	Energy discharged from battery	(Continuous)
s_t	Battery state of charge	(Continuous)
$y_t^{charge}, y_t^{discharge} \in \{0, 1\}$	Whether charging or discharging battery	(Binary)

Objective Function

$$\sum_{t=0}^{T}\left(C_t^{sell}x_t^{sell} - C_t^{buy}x_t^{buy}\right) \rightarrow max$$

Constraints

For each hour $t = 0, 1, 2, 3, \dots, T$

- <u>Power balance:</u>

$$E_t^{demand} = E_t^{solar} + x_t^{buy} + x_t^{discharge} - x_t^{charge} - x_t^{sell}$$

- <u>Battery state of charge:</u>

$$s_t = \begin{cases} s_{\text{initial}} + \eta_{\text{charge}} x_t^{charge} - \frac{x_t^{discharge}}{\eta_{\text{discharge}}} & t = 0 \\ s_{t-1} + \eta_{\text{charge}} x_t^{charge} - \frac{x_t^{discharge}}{\eta_{\text{discharge}}} & t \geq 1 \end{cases}$$

$$s_t \leq E^{cap} \quad , s_{initial} \quad given$$

- <u>Battery charge/discharge limits:</u>

$$x_t^{charge} \leq P_{max}^{charge} y_t^{charge}$$

$$x_t^{discharge} \leq P_{discharge}^{max} y_t^{discharge}$$

- <u>No simultaneous battery charging and discharging:</u>

$$y_t^{charge} + y_t^{discharge} \leq 1$$

$$y_t^{charge}, y_t^{discharge} \in \{0, 1\}$$

- <u>Grid buy/sell limits:</u>

$$x_t^{buy} \leq P_{buy}^{max}$$

$$x_t^{sell} \leq P_{sell}^{max}$$

- <u>Non-negativity:</u>

$$x_t^{buy}, x_t^{sell}, x_t^{charge}, x_t^{discharge}, s_t, y_t^{charge}, y_t^{discharge} \geq 0$$

2.2.2 Solution Methodology:

For each hour t, the formulated MILP contains 5 continuous variables and 2 binary variables. Therefore, for a 24 hour period, the formulated MILP contains 120 continuous variables and 48 binary variables.

This MILP formulation is necessary because the system contains both continuous variables (battery SOC and energy bought, sold, charged, or discharged) and binary variables (whether the battery is charging or discharging).

The objective of the model is to maximize profit (or equivalently, minimize the electricity cost) while satisfying all operational constraints

Solver and Algorithm:

The model is implemented in Python using the PuLP library, which interfaces with the CBC solver. The CBC solver then finds the optimal solution using the following steps:

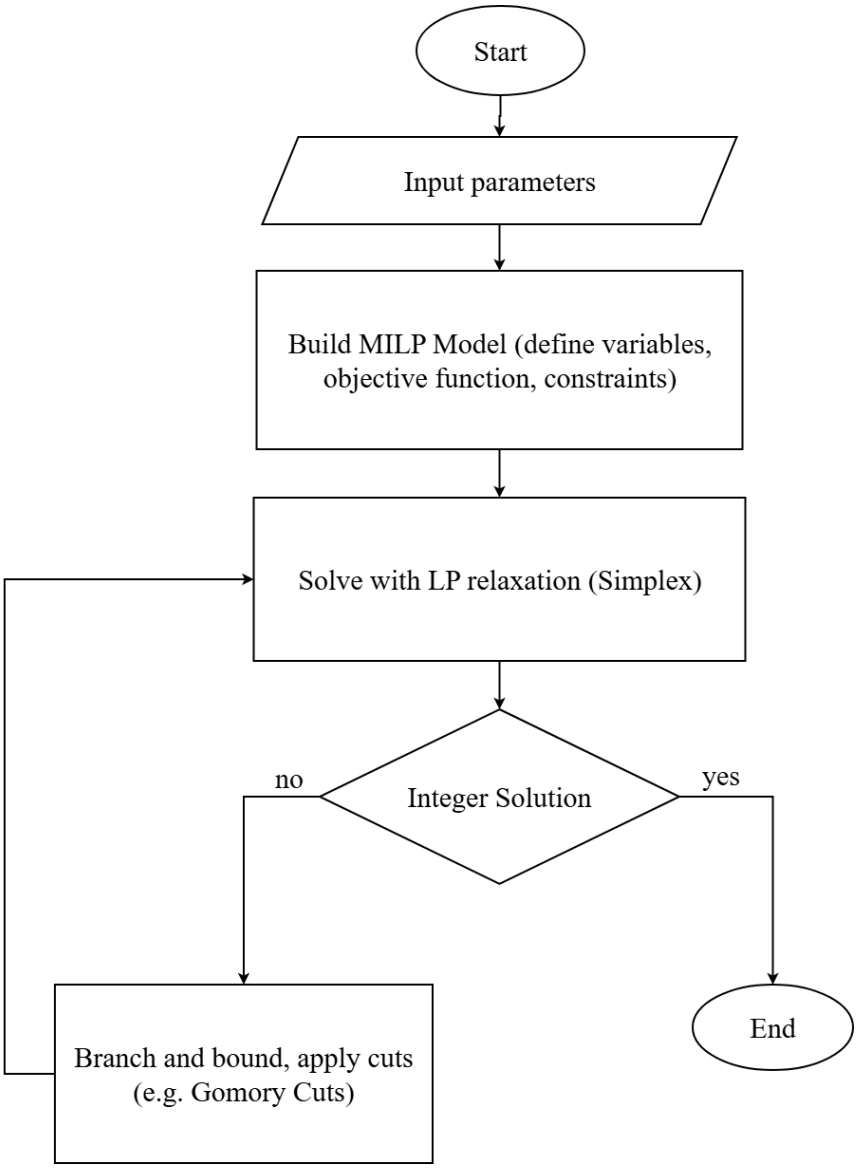


Figure 2. The solution algorithm for the linear case

LP Relaxation: All binary variables are temporarily relaxed to continuous values. The relaxed linear program is solved using the Simplex method.

Branch-and-Bound: If any binary variable in the LP solution is fractional, the solver creates two branches: one where the variable is forced to 0, and another where it is forced to 1. This process continues recursively, removing branches that cannot improve the current best solution.

Gomory Cuts: CBC automatically adds Gomory cuts, which are linear constraints that eliminate fractional solutions. These cuts work by adding a new linear constraint that cuts off the fractional solution but doesn't exclude any feasible integer solutions.

2.3. Nonlinear Programming Model:

2.3.1 Mathematical Model:

For the nonlinear case, we keep the variables and objective function and we introduce a new parameter:

k	Loss coefficient

We also adjust the battery SOC constraint to be convex and nonlinear:

- <u>Battery state of charge:</u>

$$E_t^{loss} = k \cdot \left(\frac{(x_t^{charge})^2}{P_{charge}} + \frac{(x_t^{discharge})^2}{P_{discharge}} \right)$$

$$s_t = \begin{cases} s_{\text{initial}} + \eta_{\text{charge}} x_t^{charge} - \frac{x_t^{discharge}}{\eta_{\text{discharge}}} - E_t^{loss} & t = 0 \\ s_{t-1} + \eta_{\text{charge}} x_t^{charge} - \frac{x_t^{discharge}}{\eta_{\text{discharge}}} - E_t^{loss} & t \geq 1 \end{cases}$$

$$s_t \leq E^{cap} \quad , s_{initial} \ given$$

All other constraints remain the same

2.3.2 Solution Methodology:

Adding the nonlinear constraint turns the original MILP into a convex Mixed-Integer Nonlinear Program (MINLP) with 120 continuous variables, 48 binary variables (charge/discharge mode per hour) and 24 convex nonlinear constraints for the 24 hour case

Solver and Algorithm:

The problem is solved using the GEKKO library in python. GEKKO solves it to proven global optimality using the Outer Approximation strategy and SQP as the continuous NLP solver. This combination works for MINLPs of this size.

The algorithm executes the following steps:

1. Initialization: Relax all binary variables (y_t^{charge}, $y_t^{discharge}$) to continuous variables in their bounds [0,1] creating a nonlinear problem.

2. Node processing in Branch-and-Bound tree

- Select an open node from the B&B tree.
- Solve the current node's NLP relaxation using Sequential Quadratic Programming (SQP).

3. Bounding and fathoming

- *If* the NLP is **infeasible**, *then* fathom (prune) the node.
- *If* the relaxed objective value ≥ current best integer feasible upper bound *then* fathom the node (by bound).
- *If* the solution has all integer variables integer-valued (within tolerance) *then* it's a new candidate integer solution, update the best upper bound if better.

4. Branching (if needed)

- If any integer/binary variable is **fractional** *then* select a branching variable.
- Create **two child nodes**:
    - One with the variable ≤ floor(value)
    - One with the variable ≥ ceil(value)
- Add children to the active node list.

Continue until no active nodes remain (tree exhausted). Return the best integer-feasible solution found (with objective = best upper bound).

Chapter 3: Software Implementation and Code Structure

3.1 Code Structure

```
solar-battery-optimization
    README.md
    LICENSE
    .gitignore
    src
        main.py
        time_analysis.py
        constants.py
        profiles.py
        linear.py
        non_linear.py
        charts.py
    results
        solar_vs_demand.png
        comparison.png
        linear
            (Linear)_Scenario_1_*.csv, *.png
            (Linear)_Scenario_2_*.csv, *.png
            (Linear)_sensitivity_analysis.png
        non_linear
            (Non-linear)_Scenario_1_*.csv, *.png
            (Non-linear)_Scenario_2_*.csv, *.png
            (Non-linear)_sensitivity_analysis.png
```

The optimization models are implemented in Python using a modular structure to separate data, model logic, and visualization. The codebase consists of a few focused modules:

- **constants.py**

  Purpose: Central definition of all physical and economic parameters for the 24-hour microgrid model. A single file is used so that both the linear and nonlinear formulations share identical data.

  Contents: Time *T*, grid buy price *C_buy* time-of-use tariff, solar generation *Esolar* and load demand *Edemand*, battery capacity *Ecap*, maximum charge/discharge power *Pcharge_max* and *Pdischarge_max*, round-trip efficiencies *charge_eff* and discharge_eff, initial state of charge *s0*. grid limits *Pbuy_max* and *Psell_max.*

- **profiles.py**

  Purpose: Builds 24-hour sell-price (export tariff) profiles used as input to the optimization. Ensures consistent tariff shapes across scenarios and sensitivity runs.

Function: *variable_tariff_profile(off_peak_price, peak_price)* returns a list of 24 prices. Off-peak is applied to hours 0-6 and 23; peak to hours 7-22.

- **linear.py**

Purpose: Implements the mixed-integer linear program (MILP) for the 24-hour dispatch problem. Solves for optimal grid buy/sell, battery charge/discharge, and state of charge to maximize net revenue (export revenue minus grid purchase cost).

Libraries used: *PuLP*, *CBC Coin-or Branch and Cut* (open-source MILP solver, invoked via *PULP_CBC_CMD* with *msg=False*). *pandas* (used to build the results DataFrame). *charts* (module in this project for plotting when *save_results* is *True*).

Main function: *solve_scenario(C_sell, scenario_name, results_folder, save_results=True)*.

- Inputs: *C_sell* is the 24-hour sell-price profile; *scenario_name* and *results_folder* for labelling and saving; *save_results* controls whether to write CSV and generate charts.
- Output: If *save_results* is *True*, writes CSV and calls chart functions and returns a pandas DataFrame. If *save_results* is *False*, returns a dict with exported, *actual_cost*, and *baseline_cost* (used by sensitivity and timing scripts).

- **non_linear.py**

Purpose: Implements the mixed-integer nonlinear program (MINLP) for the same dispatch problem, with an added quadratic resistive loss term in the battery state-of-charge dynamics.

Libraries used: *GEKKO* (Python package for formulation and solution of dynamic and nonlinear optimization problems). *APOPT* (nonlinear solver; selected with *m.options.SOLVER = 1* and *m.solve(disp=False)*; runs locally with *GEKKO(remote=False))*. *pandas* (results DataFrame). *charts* (same as in *linear.py* when saving results).

Main function: *solve_scenario(C_sell, scenario_name, results_folder, save_results=True*).

Inputs and outputs: Same interface as *linear.solve_scenario* for fair comparison and reuse in *main.py*, sensitivity, and timing.

➢ **charts.py**

Purpose: All plotting and visualization for the project. Uses the same data structures (DataFrames and price arrays) so that linear and nonlinear results can be compared visually.

Libraries used: *Matplotlib* (all figures). NumPy (for array operations in the comparison chart). profiles (*variable_tariff_profile* for the sensitivity analysis).

Functions:

- *solar_vs_demand_chart*: 24-hour bar/line plot of solar generation vs. load demand (input data overview).
- *scenario_chart*: hourly time series of solar, demand, buy, sell, and SOC for one scenario.
- *costs_chart*: pie chart of grid purchase cost vs. export revenue.
- *sources_chart*: stacked bars of solar, grid import, and battery discharge supplying the load, with demand overlay.
- *decision_variables_chart*: binary charge/discharge indicators over time.
- *battery_charging_chart*: charge/discharge bars and SOC trajectory.
- *comparison_chart*: grouped bar chart comparing total solar, buy, discharge, and sell for all four runs (linear/nonlinear times scenario 1/2).
- *sensitivity_chart*: for a given solver, sweeps sell price from 22 to 51 AMD/kWh (30 points), calls solve_scenario with *save_results=False* for each, and plots exported energy and actual cost vs. price.

➢ **main.py**

Purpose: Top-level script that runs the full workflow: create output directories, plot solar vs. demand, run both scenarios with both solvers, produce the comparison chart, and run the sensitivity analysis for each solver.

Libraries used: *os*, and project modules *linear*, *non_linear*, *charts*, *constants*, and *profiles* for data and tariff profiles.

Flow:

1. Create results/, results/linear/, results/non_linear/.
2. Call charts.solar_vs_demand_chart with data from constants.
3. Build scenario 1 and scenario 2 sell profiles via *variable_tariff_profile*.
4. Run linear solver for both scenarios (with CSV and charts).
5. Run nonlinear solver for both scenarios (with CSV and charts).
6. Call *charts.comparison_chart* with the four result DataFrames.
7. Call *charts.sensitivity_chart* for linear and nonlinear (each runs 30 price points, no per-scenario charts).

➢ **time_analysis.py**

Purpose: Dedicated solver timing benchmark. Runs the same 24-hour scenario multiple times with each solver, with no CSV or chart output, and reports average solve time and the ratio. Used to compare computational performance of the MILP vs. MINLP.

Libraries used: Python standard library *time* (*time.perf_counter()*), plus *linear*, *non_linear*, and *profiles* (no charts).

Logic: *N_RUNS = 10* (number of runs per solver).

1. Build one price profile with *variable_tariff_profile*.
2. Run *linear.solve_scenario* and *nonlinear.solve_scenario* with *save_results=False* 10 times,
3. Record each elapsed time and compute the mean.
4. Print average time in seconds for each solver and the ratio (nonlinear over linear).

Chapter 4: Case Study: Commercial Building in Yerevan

4.1 System parameters

We consider the following system:

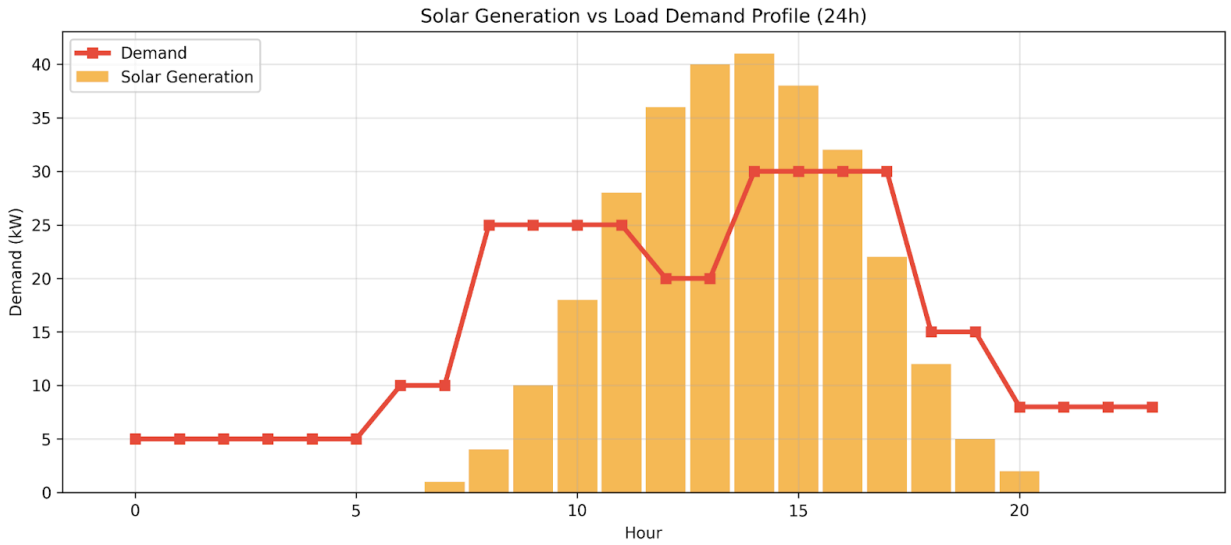
- 30 kWh lithium battery, 12 kW charge/discharge power, η = 95 %
- Grid tariffs: buy 38 AMD (00–07) / 52 AMD (rest) and 2 export price scenarios:
  - Scenario 1: 22 AMD
  - Scenario 2: 35 AMD (00–07) / 48 AMD (rest)
- Total daily load of 372 kWh
- Total rooftop solar production of 289 kWh
- Max grid buy power in one hour: 60kW, sell power: 30kW
- Initial battery state of charge: 15 kWh

The given parameters are:

E^{cap}	30 kWh	P^{max}_{buy}	60 kW
P^{max}_{charge}, $P^{max}_{discharge}$	12 kW	η_{charge}, $\eta_{discharge}$	0.95
P^{max}_{sell}	30 kW	s_0	15 kWh

t	E^{solar}_t	E^{demand}_t	P^{buy}_t	P^{sell}_t (Scenario 1)	P^{sell}_t (Scenario 2)
0	0	5	38	22	35
1	0	5	38	22	35
2	0	5	38	22	35
3	0	5	38	22	35
4	0	5	38	22	35
5	0	5	38	22	35
6	0	10	38	22	35
7	1	10	38	22	35
8	4	25	38	22	48
9	10	25	52	22	48
10	18	25	52	22	48
11	28	25	52	22	48
12	36	20	52	22	48
13	40	20	52	22	48

14	41	30	52	22	48
15	38	30	52	22	48
16	32	30	52	22	48
17	22	30	52	22	48
18	12	15	52	22	48
19	5	15	52	22	48
20	2	8	52	22	48
21	0	8	52	22	48
22	0	8	52	22	48
23	0	8	52	22	35



For $t = 0, 1, 2, \ldots, 23$, we need to find the optimal solution of the mathematical model described earlier

The objective function value represents the optimized net cost of electricity after the model schedules charging, discharging, buying, and selling. A higher net cost indicates a more economically efficient operation. The Amount Saved is computed by comparing this optimized cost to the baseline cost of running the building without solar and battery support.

$$Baseline\ Cost = \sum_{t=0}^{T} E_t^{demand} P_t^{buy} = 18672\ AMD$$

$$Amount\ Saved = Baseline\ Cost - Actual\ Cost$$

4.2 Results

Linear Case - Scenario 1: 22 AMD

t	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$
0	5	0	0	0	15	0	0
1	5	0	0	0	15	0	0
2	5	0	0	0	15	0	0
3	5	0	0	0	15	0	0
4	5	0	0	0	15	0	0
5	17	0	12	0	26.4	1	0
6	13.79	0	3.79	0	30	1	0
7	0	0	0	9	20.53	0	1
8	20.5	0	0	0.5	20	0	1
9	3	0	0	12	7.37	0	1
10	0	0	0	7	0	0	1
11	0	3	0	0	0	0	0
12	0	16	0	0	0	0	0
13	0	9.42	10.58	0	10.05	1	0
14	0	0	11	0	20.5	1	0
15	0	0	8	0	28.1	1	0
16	0	0	2	0	30	1	0
17	0	0	0	8	21.58	0	1
18	0	0	0	3	18.42	0	1
19	0	0	0	10	7.89	0	1
20	0	0	0	6	1.58	0	1
21	6.5	0	0	1.5	0	0	1
22	8	0	0	0	0	0	0
23	8	0	0	0	0	0	0

Exported Amount: 28.4 kWh

Value of the objective function: -3774.74

Actual Cost: 3774.74 AMD

Amount Saved = Baseline Cost - Actual Cost = 18672 -3774.74= 14897.26 AMD

Linear Case - Scenario 2: 35 or 48 AMD

t	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$
0	5	0	0	0	15	0	0
1	5	0	0	0	15	0	0
2	5	0	0	0	15	0	0
3	5	0	0	0	15	0	0
4	5	0	0	0	15	0	0
5	8.79	0	3.79	0	18.6	1	0
6	22	0	12	0	30	1	0
7	0	0	0	9	20.53	0	1
8	20.5	0	0	0.5	20	0	1
9	3	0	0	12	7.37	0	1
10	0	0	0	7	0	0	1
11	0	3	0	0	0	0	0
12	0	16	0	0	0	0	0
13	0	20	0	0	0	0	0
14	0	11	0	0	0	0	0
15	0	8	0	0	0	0	0
16	0	2	0	0	0	0	0
17	8	0	0	0	0	0	0
18	3	0	0	0	0	0	0
19	10	0	0	0	0	0	0
20	6	0	0	0	0	0	0
21	8	0	0	0	0	0	0
22	8	0	0	0	0	0	0
23	8	0	0	0	0	0	0

Exported Amount: 60.0 kWh

Value of the objective function: -3002.00 AMD

Actual Cost: -3002 AMD

Amount Saved = Baseline Cost - Actual Cost = 18672 - 3002 = 15670 AMD

Nonlinear Case - Scenario 1: 22 AMD

We will consider the same cases studied in the linear case for $k = 0.012$

t	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$
0	5	0	0	0	15	0	0
1	5	0	0	0	15	0	0
2	5	0	0	0	15	0	0
3	5	0	0	0	15	0	0
4	12.96	0	7.96	0	22.5	1	0
5	5	0	0	0	22.5	0	0
6	17.96	0	7.96	0	30	1	0
7	1.9	0	0	7.1	22.47	0	1
8	13.9	0	0	7.1	14.94	0	1
9	7.9	0	0	7.1	7.42	0	1
10	0	0	0	7	0	0	1
11	0	0	3	0	2.84	1	0
12	0	8.8	7.2	0	9.63	1	0
13	0	12.8	7.2	0	16.42	1	0
14	0	3.8	7.2	0	23.21	1	0
15	0	0.8	7.2	0	30	1	0
16	0	2	0	0	30	0	0
17	2.93	0	0	5.07	24.63	0	1
18	0	0	0	3	21.47	0	1
19	4.93	0	0	5.07	16.1	0	1
20	0.93	0	0	5.07	10.73	0	1
21	2.93	0	0	5.07	5.37	0	1
22	2.93	0	0	5.07	0	0	1
23	8	0	0	0	0	0	0

Exported Amount: 28.19 kWh

Value of the objective function: -3801.52 AMD

Actual Cost: 3801.52 AMD

Amount Saved = Baseline Cost - Actual Cost = 18672 - 3801.52 = 14870.48 AMD

Nonlinear Case - Scenario 2: 35 or 48 AMD

t	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$
0	12.96	0	7.96	0	22.5	1	0
1	12.96	0	7.96	0	30	1	0
2	5	0	0	0	30	0	0
3	5	0	0	0	30	0	0
4	5	0	0	0	30	0	0
5	5	0	0	0	30	0	0
6	10	0	0	0	30	0	0
7	6.16	0	0	2.84	27	0	1
8	18.16	0	0	2.84	24	0	1
9	12.16	0	0	2.84	21	0	1
10	4.16	0	0	2.84	18	0	1
11	0	3	0	0	18	0	0
12	0	16	0	0	18	0	0
13	0	20	0	0	18	1	0
14	0	11	0	0	18	0	0
15	0	8	0	0	18	0	0
16	0	2	0	0	18	0	0
17	5.16	0	0	2.84	15	0	1
18	0.16	0	0	2.84	12	0	1
19	7.16	0	0	2.84	9	0	1
20	3.16	0	0	2.84	6	0	1
21	5.16	0	0	2.84	3	0	1
22	5.16	0	0	2.84	0	0	1
23	8	0	0	0	0	0	0

Exported Amount: 60.0 kWh

Value of the objective function: -3011.06 AMD

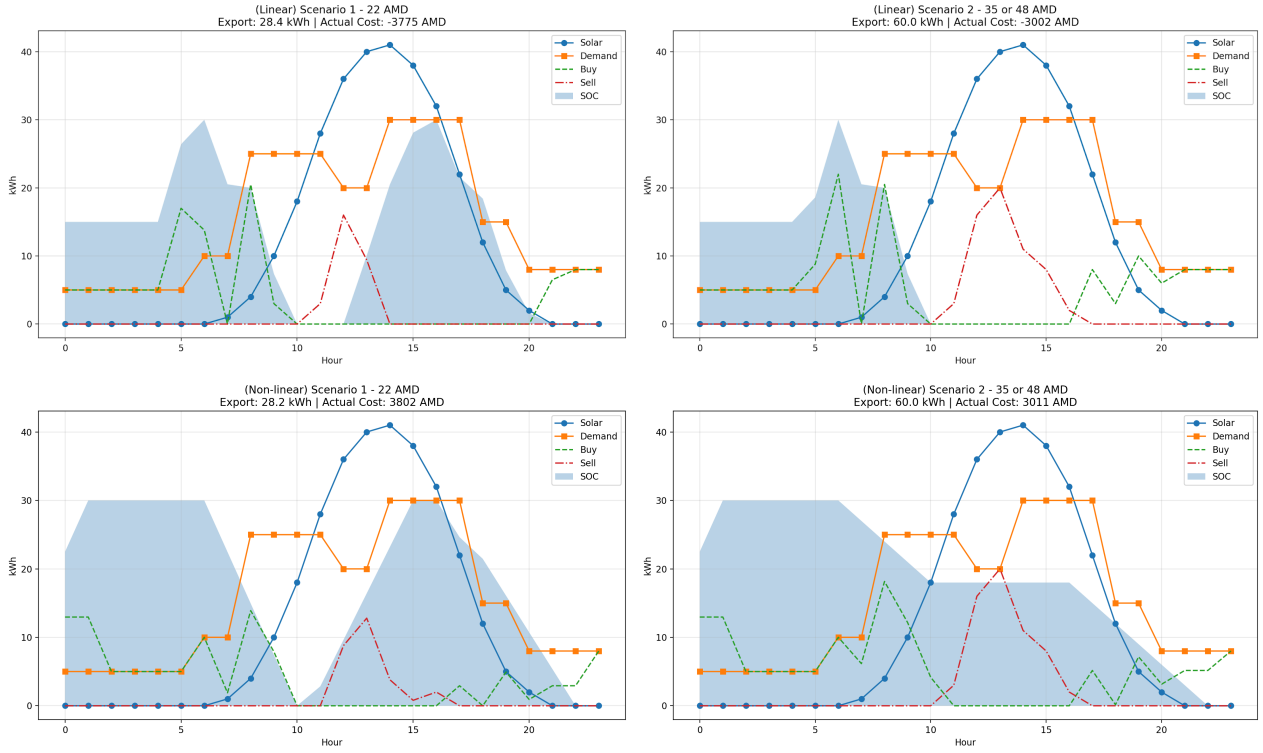
Actual Cost: 3011.06 AMD

Amount Saved = Baseline Cost - Actual Cost = 18672 - 3011.06 = 15660.94 AMD

4.3 Discussion

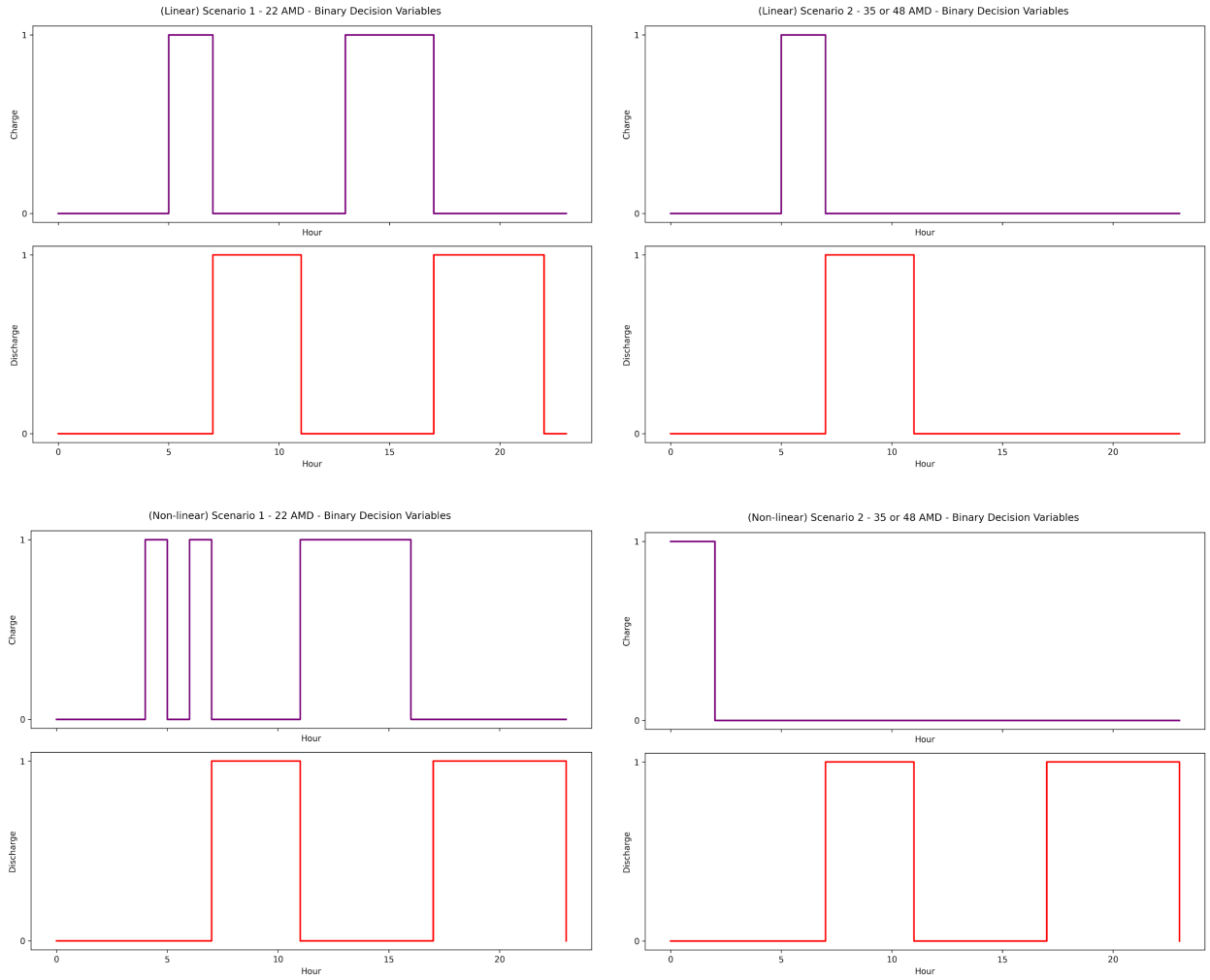
4.3.1 Interpretation of results

We compare two models (Linear MILP and Nonlinear MINLP) on two export price scenarios. The purpose of comparing these two formulations is to evaluate how physical system losses influence the optimal operating strategy of the microgrid. While the linear MILP formulation assumes constant efficiencies, the nonlinear MINLP model incorporates quadratic loss functions that better represent real battery and inverter behavior. This allows the study to examine whether accounting for nonlinear losses significantly changes the operational schedule or the economic performance of the system.



In scenario 2, the model prioritizes selling over charging the battery. The model exclusively sells during peak hours.

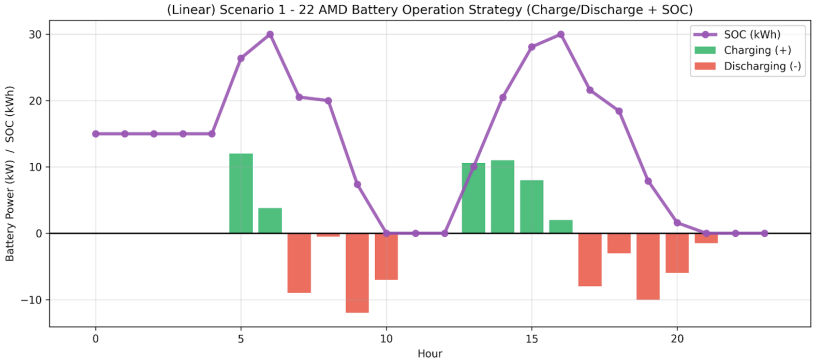
Compared to the linear case, in scenario 2, the battery SOC decreases slowly throughout the day. Whereas in other scenarios, the battery completely discharges at hour 10.

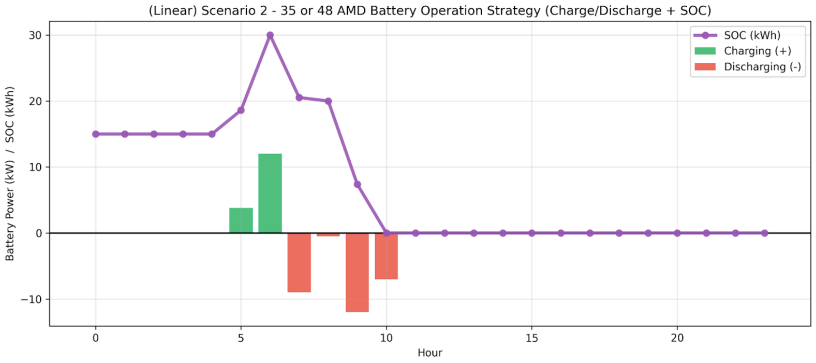


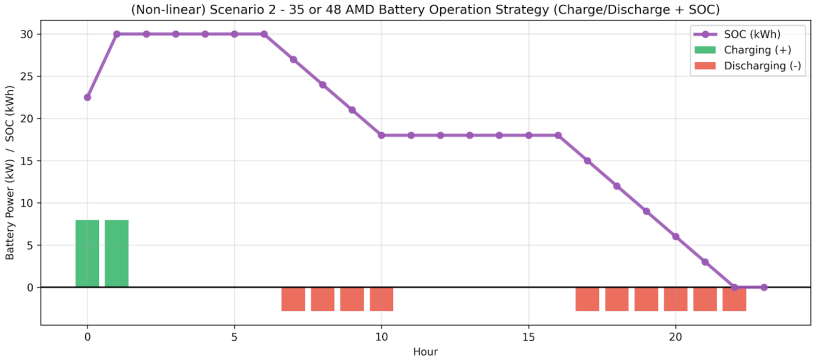
In scenario 1, the model charges the battery during hours when electricity is cheap or when solar generation is high. When solar output drops, the model discharges the battery to reduce grid purchases. This behaviour confirms that the optimization correctly prioritizes low-cost charging and high-value discharging to maximize overall savings. This charging-discharging pattern reflects the fundamental principle of energy arbitrage in time-of-use electricity markets. By storing energy during low-price periods and discharging during high-price periods, the battery effectively shifts energy consumption in time. Such behaviour demonstrates that the optimization framework successfully captures the economic incentives embedded in the tariff structure and translates them into an optimal operational schedule.

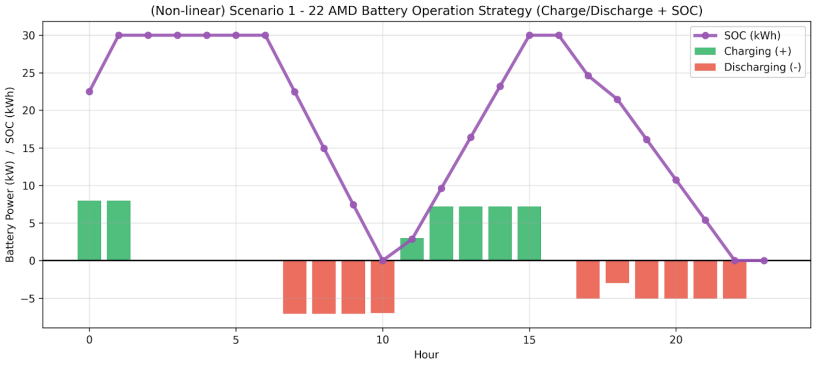
In scenario 2, the model charges the battery only during non-peak hours. During peak hours, unlike in the first scenario, the optimizer does not charge the battery at all. Instead, it sells almost all excess solar energy directly to the grid because the high selling price makes exporting more profitable than storing the energy. This behavior is confirmed by the binary variables showing that the battery neither charges or discharges starting from hour 11 in the linear case and during hours 11-16 in the non linear case, meaning all exported energy comes from real-time solar

production rather than stored energy during that time. This result highlights an important operational trade-off between storage and direct export. When export prices are sufficiently high, the opportunity cost of storing energy increases. Instead of storing excess photovoltaic generation for later use, the optimization model prefers immediate export because the financial return from selling electricity exceeds the potential benefit of future self-consumption. This demonstrates how tariff structures directly influence the role of battery storage in distributed energy systems.

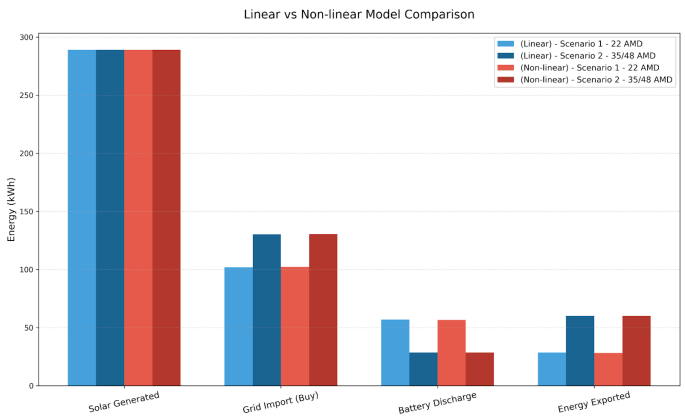




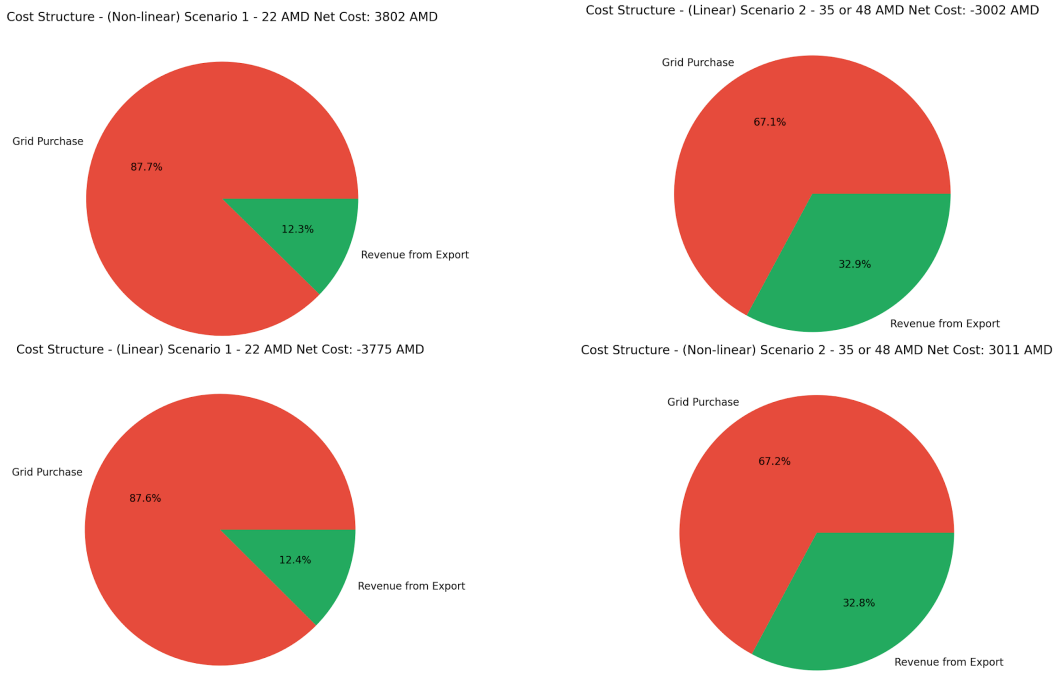




The two tariff scenarios show a clear relationship between the selling price and the optimal export behavior of the microgrid.

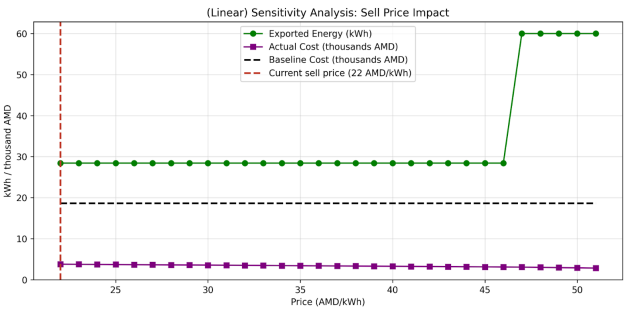


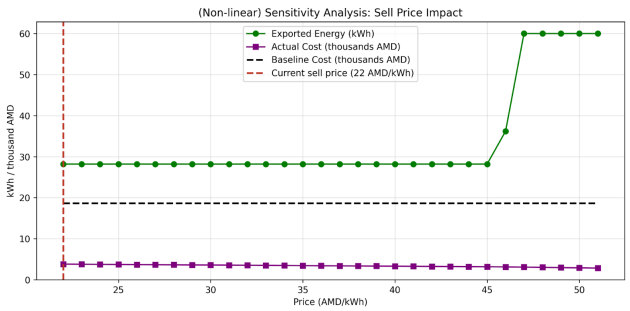
When the selling price is 22 AMD/kWh (Scenario 1), the optimizer exports 28 kWh to the grid. When the selling price increases to 35 or 48 AMD/kWh (Scenario 2), the exported energy rises to 60.0 kWh, indicating that higher selling prices cause the model to prioritize exporting energy rather than self-consumption. This behaviour illustrates the strong sensitivity of optimal microgrid operation to electricity pricing policies. Even relatively small increases in export tariffs can significantly alter the energy flow patterns within the system. From a policy perspective, this suggests that feed-in tariffs and export compensation mechanisms play a crucial role in determining whether distributed energy resources are primarily used for self-consumption or for grid supply. In scenario 2, the optimizer buys more energy from the grid at non-peak hours, charges the battery and then sells at peak hours when it is more profitable.



In Scenario 2, both models yield similar operating strategies and therefore the same gross exported energy (60 kWh). However, the nonlinear model explicitly accounts for quadratic inverter and battery losses. These physical losses represent energy that is consumed internally and does not reach the building load or the grid. To satisfy the hourly energy balance under the presence of these additional losses, the nonlinear model purchases more electricity from the grid

compared to the linear case. This small increase in grid purchases directly demonstrates the real economic penalty of inverter and battery inefficiency that is completely invisible in traditional linear formulations. This difference is seen through the 0.1% shift in the cost structure chart between the linear and nonlinear cases in both export-price scenarios. Although the numerical difference between the linear and nonlinear results appears small, the inclusion of nonlinear losses improves the physical realism of the model. Real batteries and inverters exhibit efficiency losses that depend on power levels, temperature, and operating conditions. By incorporating these nonlinear characteristics, the MINLP formulation provides a more accurate representation of system behaviour and prevents the optimizer from exploiting unrealistic efficiency assumptions.





Increasing the sell price leads to higher export levels. Once the sell price becomes greater than the buy price, exporting becomes more profitable than storing or self-consuming energy, resulting in a rise in exported energy.

Overall, the results demonstrate that the optimal management of photovoltaic generation and battery storage depends strongly on both the tariff structure and the physical characteristics of the energy storage system. The comparison between linear and nonlinear formulations confirms that including realistic loss mechanisms slightly reduces economic performance but produces a more reliable estimate of system operation. These findings highlight the importance of accurate system modeling when evaluating the economic feasibility of grid-connected microgrids.

4.3.2 Model Assumptions and Limitations

The optimization framework presented in this thesis relies on several simplifying assumptions that allow the problem to remain computationally tractable while still capturing the key operational characteristics of the PV battery system.

First, photovoltaic generation is assumed to be perfectly known in advance. In practice, solar generation is subject to forecasting uncertainty due to weather variability. The current model

therefore represents an idealized scheduling environment where generation profiles are deterministic.

Second, electricity demand is treated as a fixed and known load profile. Real buildings often exhibit unpredictable consumption patterns influenced by occupant behavior and environmental conditions. Incorporating demand uncertainty would require unpredictable or robust optimization approaches.

Another simplifying assumption concerns battery performance. Although nonlinear losses are incorporated in the MINLP model, other physical effects such as temperature dependence, battery degradation, and aging are not explicitly modeled. In real energy storage systems, repeated cycling gradually reduces battery capacity and efficiency over time.

Despite these limitations, the proposed model captures the primary economic interactions between photovoltaic generation, battery storage, and time-varying electricity tariffs. Therefore, it provides a useful approximation for evaluating optimal energy management strategies in grid-connected microgrid systems.

4.3.3 Computational Performance of MILP vs MINLP

The computational complexity of the optimization problem differs significantly between the linear MILP formulation and the nonlinear MINLP formulation. Linear mixed-integer problems can typically be solved efficiently using branch-and-bound algorithms combined with linear programming relaxations. In contrast, nonlinear mixed-integer problems introduce additional challenges because nonlinear constraints make the feasible region more complex.

To assess the computational trade-off between the linear and nonlinear formulations of the microgrid dispatch problem, a timing benchmark was conducted. The goal was to compare the raw solve time of each approach under identical conditions, isolating the solver performance from any I/O or post-processing overhead.

Both formulations were evaluated on the same 24-hour dispatch scenario: a flat export tariff of 22 AMD/kWh with the standard solar, demand, and grid price profiles defined in the model. To obtain statistically stable results, each solver was run 10 times consecutively, and the mean execution time was reported. The benchmark measured only the optimization solve itself, no CSV export, chart generation, or console output to ensure a fair comparison of solver performance.

Model	Time (avg of 10)
Linear (PuLP/CBC)	0.103s
Nonlinear (GEKKO)	0.485s

The benchmark was executed on a standard desktop environment. The linear solver (PuLP/CBC) completed a single 24-hour scenario in approximately 0.1 seconds on average, while the nonlinear solver (GEKKO/APOPT) required approximately 0.5 seconds per scenario. The nonlinear formulation was thus around 5 times slower than the linear one for this problem size.

The MILP model solved relatively quickly due to the linear structure of the objective function and constraints. The solver was able to identify optimal charging and discharging schedules within a short computation time.

The MINLP model required greater computational effort due to the presence of quadratic loss functions. Nonlinear constraints increase the difficulty of the optimization problem because the solver must evaluate nonlinear relationships while simultaneously handling integer decision variables.

However, the nonlinear formulation provides a more realistic representation of system behavior. Although the computational burden is higher, the resulting schedules better reflect physical efficiency losses within the battery and inverter system.

This trade-off between computational efficiency and modeling accuracy is a common consideration in energy system optimization. Linear models are often preferred for large-scale planning problems, while nonlinear formulations are valuable when more detailed representation of physical processes is required.

4.3.4 Practical Implications

These results provide several insights for the design and operation of photovoltaic battery systems in grid-connected buildings.

First, the optimization results demonstrate that battery storage can effectively reduce electricity costs by shifting energy consumption from high-price periods to low-price periods. This confirms the economic value of battery storage under time-of-use tariff structures.

Second, the results highlight the importance of electricity export pricing policies. When export prices increase, the optimization model prioritizes exporting excess solar energy instead of

storing it for later use. This indicates that regulatory frameworks and feed-in tariffs can significantly influence the operational role of distributed energy resources.

Third, the comparison between linear and nonlinear models shows that ignoring nonlinear efficiency losses may slightly overestimate the economic benefits of battery storage. Although the difference in total cost is relatively small, incorporating realistic loss models leads to more accurate predictions of system performance.

These findings are relevant for energy system planners, building managers, and policymakers who are evaluating the economic viability of distributed renewable energy systems. By using optimization-based energy management strategies, buildings can improve the utilization of locally generated renewable energy while reducing reliance on grid electricity during peak demand periods.

Conclusion

This thesis investigated the optimal daily energy management of a grid-connected photovoltaic battery system under time-of-use electricity tariffs. The objective was to determine the optimal hourly schedule for purchasing electricity from the grid, exporting surplus solar energy, and charging or discharging a lithium-ion battery in order to minimize the total electricity cost of a commercial building.

Two mathematical optimization models were developed and compared. The first model formulated the problem as a mixed-integer linear program (MILP) assuming constant battery efficiency. The second model extended the formulation to a convex mixed-integer nonlinear program (MINLP) by incorporating quadratic power-dependent loss terms representing realistic inverter and battery losses. Both models were implemented in Python using open-source optimization tools: PuLP with the CBC solver for the linear case, and GEKKO with the APOPT solver for the nonlinear case.

A case study based on a representative commercial building in Yerevan was used to evaluate the models under two different export compensation scenarios. The results show that the optimization framework significantly reduces electricity costs compared to a baseline grid-only operation. The optimal schedules demonstrate typical energy management behavior: the battery charges during periods of low electricity prices or excess solar generation and discharges during high-price periods to reduce grid purchases.

The comparison between the linear and nonlinear models highlights the importance of accounting for realistic system losses. While the linear model slightly overestimates achievable savings due to the assumption of constant efficiency, the nonlinear formulation provides a more accurate representation of real system behavior. Nevertheless, the linear model remains useful as a computationally efficient approximation and as a benchmark for the nonlinear case.

The developed optimization framework demonstrates how mathematical programming techniques can support energy management decisions for distributed energy systems. The results suggest that combining photovoltaic generation with battery storage can substantially improve the economic performance of commercial buildings operating under time-of-use tariffs.

References

1. APOPT Solver. (n.d.). Retrieved November 26, 2025, from https://github.com/APMonitor/apopt

2. Bean, R., & Khan, H. (2018). Using solar and load predictions in battery scheduling at the residential level. *Conference Paper / Journal Name* [please insert journal or conference], 2018.

3. CBC (COIN-OR Branch and Cut) Solver. (n.d.). Retrieved November 26, 2025, from https://github.com/coin-or/Cbc

4. COIN-OR. (n.d.). *PuLP: A linear programming toolkit for Python*. Retrieved March 1, 2026, from https://coin-or.github.io/pulp/main/includeme.html

5. Cornell University. (2021). *Branch and bound (BB) for MINLP*. Retrieved March 1, 2026, from https://optimization.cbe.cornell.edu/index.php?title=Branch_and_bound_(BB)_for_MINLP

6. Cornell University. (2021). *Simplex algorithm*. Retrieved March 1, 2026, from https://optimization.cbe.cornell.edu/index.php?title=Simplex_algorithm

7. Electric Networks of Armenia. (n.d.). Retrieved November 22, 2025, from https://www.ena.am/Info.aspx?id=11

8. GeeksforGeeks. (n.d.). *Introduction to branch and bound*. Retrieved March 1, 2026, from https://www.geeksforgeeks.org/dsa/introduction-to-branch-and-bound-data-structures-and-algorithms-tutorial/

9. GeeksforGeeks. (n.d.). *Introduction to Pandas in Python*. Retrieved March 1, 2026, from https://www.geeksforgeeks.org/pandas/introduction-to-pandas-in-python/

10. Gekko Development Team. (n.d.). *Gekko overview*. Retrieved March 1, 2026, from https://gekko.readthedocs.io/en/latest/overview.html

11. Hexaly. (n.d.). *Branch and cut algorithms*. Retrieved March 1, 2026, from https://www.hexaly.com/algorithms/branch-and-cut

12. Hovhannisyan, I. (2025). *Mathematical programming* [Lecture notes]. National Polytechnic University of Armenia.

13. Hunter, J., et al. (n.d.). *Matplotlib tutorials*. Matplotlib.org. Retrieved March 1, 2026, from https://matplotlib.org/stable/tutorials/index.html

14. Kleissl, J., Nottrott, A., & Washom, B. (2013). Energy dispatch schedule optimization and cost-benefit analysis for grid-connected photovoltaic-battery storage systems. *Renewable Energy, 55*, 230–240. https://doi.org/10.1016/j.renene.2012.12.036

15. Parkinson, A. R., Balling, R., & Hedengren, J. D. (2018). *Optimization methods for engineering design* (2nd ed.). Brigham Young University.

16. Python Software Foundation. (n.d.). Retrieved March 1, 2026, from https://www.python.org/

17. W3Schools. (n.d.). Retrieved March 1, 2026, from https://www.w3schools.com/